Etapa 3: Inițializarea proiectului (Vite și React)

Tehnologii Web · Proiect de semestru · Etapa 3

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Inginerie Industrială și Robotică
Disciplina: Tehnologii Web, proiect de semestru
Anul universitar: 2026–2027
Coordonatori de proiect: Emilia PĂSĂRIN, Alexandru RIZEA, Andrei-Alexandru STAICU

Enunțul etapei

Să se creeze, cu Vite, un proiect React în folderul `frontend/` al repository-ului și să se ruleze aplicația cu serverul de dezvoltare. Fișierele demonstrative generate de Vite se elimină, stilurile din Etapa 1 se reutilizează integral, iar datele din Etapa 2 se mută în componenta principală. Antetul afișează valori calculate din date; lista se afișează deocamdată ca text brut (JSON), componentele care o desenează fiind realizate în Etapa 4.

De acum înainte, proiectul se dezvoltă ca o aplicație web reală: pachetele se instalează cu npm, aplicația rulează pe un server local, iar interfața se scrie în JSX. La fiecare salvare a unui fișier, pagina se actualizează automat, fără reîncărcare manuală.

La finalul etapei veți putea:

- instala dependențele unui proiect cu npm și explica rolul fișierului `package.json`;
- crea și porni un proiect React cu Vite;
- scrie o componentă simplă în JSX;
- afișa în interfață valori calculate din date;
- verifica faptul că proiectul se construiește fără erori (`npm run build`).

Noțiuni introductive

npm și package.json

npm este managerul de pachete pentru JavaScript. Fișierul `package.json` conține lista dependențelor și comenzile proiectului (de exemplu `dev` și `build`). Comanda `npm install` descarcă dependențele în folderul `node_modules/`. Acest folder nu se publică niciodată pe Git, deoarece poate fi recreat oricând din `package.json`; fișierul `.gitignore` generat de Vite îl exclude automat.

Vite

Vite rulează aplicația în timpul dezvoltării și o împachetează pentru publicare. Comanda `npm run dev` pornește un server local, de obicei la adresa `http://localhost:5173`, care transmite modificările către browser la fiecare salvare. Pagina nu se mai deschide prin dublu clic pe fișier, deoarece fișierele se încarcă unele pe altele prin `import`, iar browserul nu permite acest lucru pentru fișiere locale.

JSX

JSX este o sintaxă care permite scrierea de HTML direct în JavaScript. Diferențele față de HTML care produc cele mai multe erori la început sunt prezentate în tabelul 3.1.

Tabelul 3.1: Diferențele dintre HTML și JSX

În HTML	În JSX	Motiv
`class="panel"`	`className="panel"`	`class` este cuvânt rezervat în JavaScript
`<input>`	`<input />`	orice etichetă trebuie închisă
text fix	`{taskuri.length}`	între acolade se scrie orice expresie JavaScript
mai multe etichete la rădăcină	un singur părinte sau fragmentul `<>...</>`	o componentă întoarce o singură valoare
`<!-- comentariu -->`	`{/* comentariu */}`	comentariile HTML nu funcționează în JSX

Componenta

O **componentă** este o funcție care întoarce JSX și al cărei nume începe cu literă mare. `App` este componenta principală; ea este montată în pagină de fișierul `main.jsx`, în elementul `<div id="root">` din `index.html` (figura 3.1).

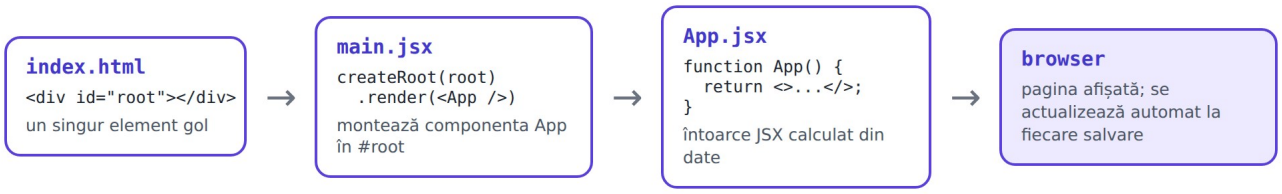


Fig. 3.1: Cum ajunge componenta App pe ecran

Adaptarea la tema proprie

Structura proiectului este identică pentru toți studenții; diferă doar denumirile și datele (tabelul 3.2).

Tabelul 3.2: Denumirile pe tema proprie

Element	TaskFlow	BookShelf	WatchList	Tema proprie
titlul paginii	TaskFlow	BookShelf	WatchList	
array-ul din App.jsx	taskuri	carti	filme	
valoarea calculată în antet	elementele active	cărțile necitite	filmele de văzut	
textul contorului	3 task-uri, dintre care 2 active	12 cărți, dintre care 5 necitite	8 filme, dintre care 3 de văzut	

Mediul de lucru

Este necesar **Node.js**, versiunea 20 sau mai nouă, instalat de pe nodejs.org (varianta *LTS*). Programul de instalare include și npm. Verificarea se face în terminal:

```
node -v
npm -v
```

Dacă una dintre comenzi nu este recunoscută, se instalează Node.js și se redeschide terminalul. Comenzile din acest document sunt aceleași în terminalul din VS Code, în Git Bash și în PowerShell.

Desfășurarea lucrării

Pașii se execută în ordinea de mai jos. Serverul de dezvoltare rămâne pornit începând cu Pasul 4; comenzile ulterioare se rulează într-un al doilea terminal.

Pasul 1: Mutarea fișierelor din etapele anterioare

În rădăcina repository-ului se creează folderul `mockup/`, în care se mută `index.html`, `style.css` și fișierul JavaScript din Etapa 2. Fișierele rămân în istoric, dar nu mai interferează cu proiectul nou:

```
mkdir mockup
mv index.html style.css taskuri.js mockup/
```

Pasul 2: Crearea proiectului React

Din rădăcina repository-ului (nu din `mockup/`) se rulează:

```
npm create vite@latest frontend -- --template react
```

Comanda pune câteva întrebări, iar setul lor diferă de la o versiune la alta a Vite. Selecția se face cu săgețile, iar confirmarea cu **Enter** (figura 3.2 și figura 3.3).

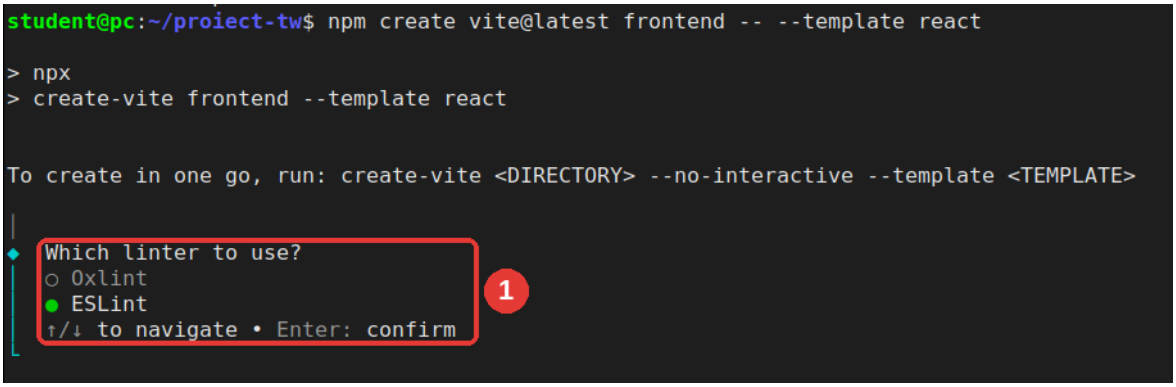


Fig. 3.2: Alegerea linterului

1. *Which linter to use?* Se alege **ESLint** (săgeata în jos, apoi Enter). ESLint este standardul, cu extensie pentru VS Code și documentație amplă. Linterul semnalează greșelile în editor, înainte să apară în browser.

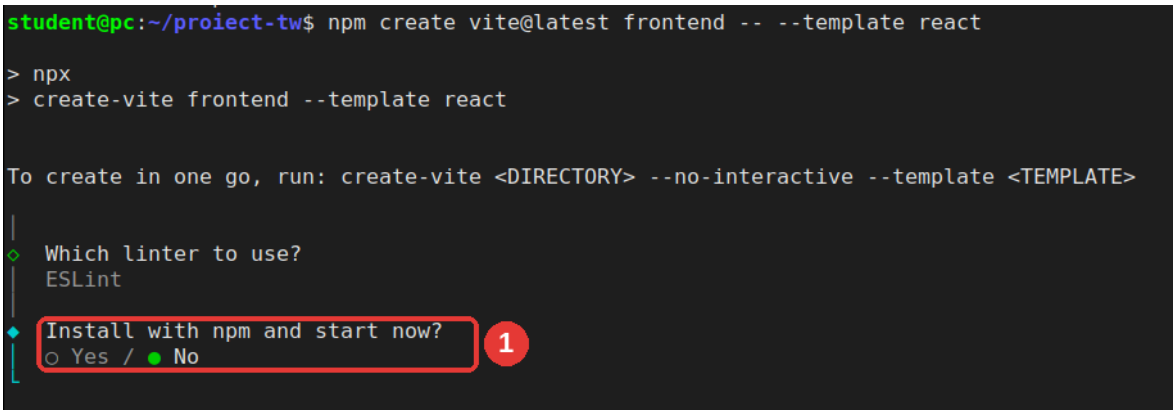


Fig. 3.3: Instalarea și pornirea se fac manual

1. *Install with npm and start now?* Se alege **No** (săgeata spre dreapta, apoi Enter). Instalarea și pornirea se fac la pasul următor, pentru a înțelege rolul fiecărei comenzi.

La final, Vite afișează comenzile de rulat (figura 3.4).

```
student@pc:~/proiect-tw$ npm -v
10.9.7
student@pc:~/proiect-tw$ mkdir mockup
student@pc:~/proiect-tw$ mv index.html style.css taskuri.js mockup/
student@pc:~/proiect-tw$ ls
README.md  mockup
student@pc:~/proiect-tw$ npm create vite@latest frontend -- --template react

> npx
> create-vite frontend --template react


To create in one go, run: create-vite <DIRECTORY> --no-interactive --template <TEMPLATE>

│
◇  Which linter to use?
│  ESLint
│
◇  Install with npm and start now?
│  No
│
◇  Scaffolding project in /home/student/proiect-tw/frontend...
│
└  Done. Now run:

  cd frontend
  npm install
  npm run dev

student@pc:~/proiect-tw$
```

Fig. 3.4: Proiectul a fost creat în folderul frontend

Pasul 3: Instalarea pachetelor

```
cd frontend
npm install
```

```
student@pc:~/proiect-tw/frontend$ npm install

added 145 packages, and audited 146 packages in 3s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
student@pc:~/proiect-tw/frontend$
```

Fig. 3.5: Instalarea dependențelor cu npm install

Comanda descarcă pachetele în `frontend/node_modules/`. Numărul exact de pachete diferă în funcție de versiune. Mesajul `found 0 vulnerabilities` arată că nu s-au găsit probleme cunoscute de securitate.

Pasul 4: Pornirea serverului de dezvoltare

```
npm run dev
```

```
  VITE v8.3.1  ready in 318 ms

  ➜  Local:   http://localhost:5173/  1
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Fig. 3.6: Serverul de dezvoltare pornit

1. Adresa aplicației. Se deschide în browser (Ctrl + clic pe adresă în terminalul VS Code). Serverul rămâne pornit cât timp comanda rulează; se oprește cu **Ctrl + C**.

În browser apare pagina demonstrativă generată de Vite (figura 3.7). Aspectul ei diferă în funcție de versiune.

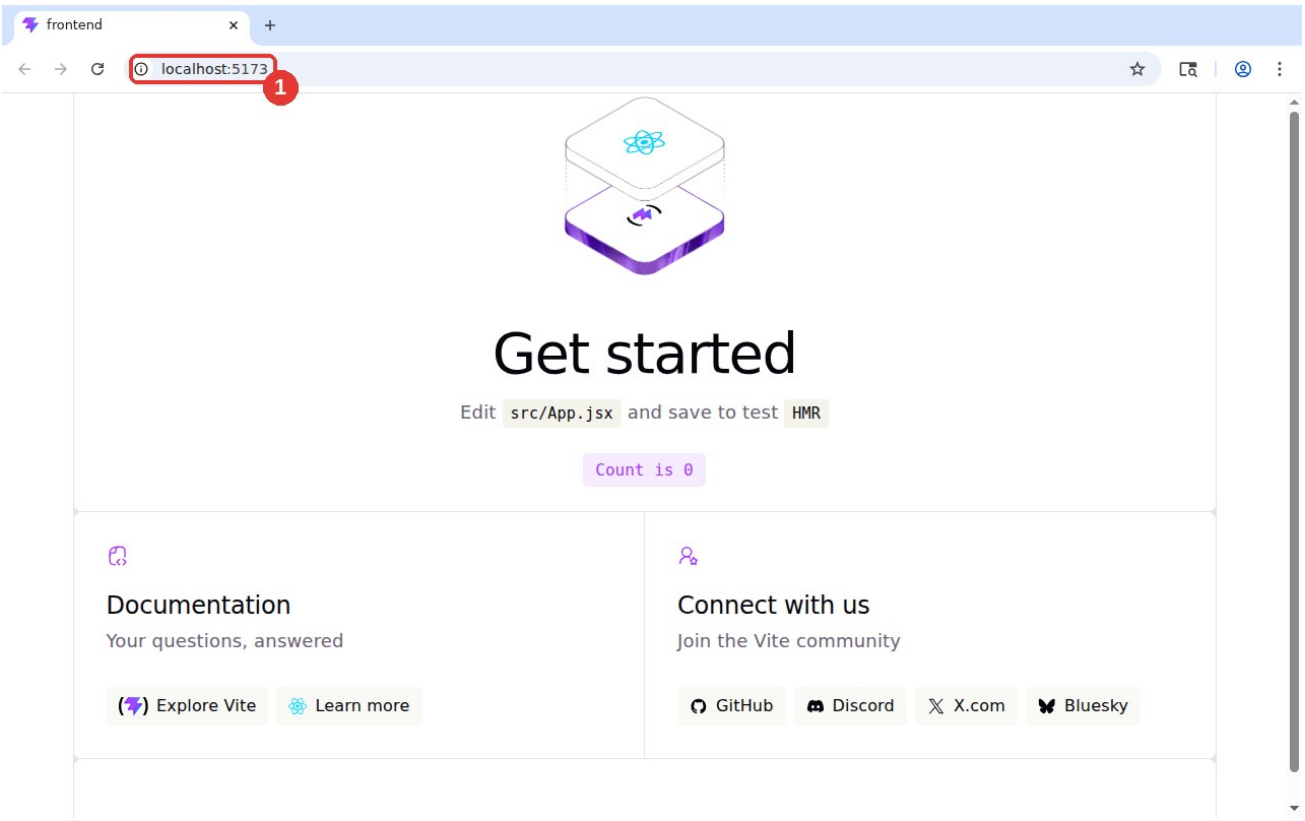


Fig. 3.7: Pagina demonstrativă generată de Vite

1. Aplicația rulează pe serverul local, la adresa `localhost:5173`, nu direct de pe disc.

Pasul 5: Eliminarea fișierelor demonstrative

Pagina demonstrativă nu se folosește în continuare. Se șterg fișierele indicate în tabelul 3.3. În VS Code: clic dreapta pe fișier sau pe folder, apoi *Delete*; pentru conținutul unui fișier: Ctrl + A, Delete, apoi salvare cu Ctrl + S.

Tabelul 3.3: Fișierele demonstrative care se elimină

Ce se șterge	Locație	Motiv
folderul `assets/`, cu tot conținutul	`frontend/src/`	siglele și imaginile paginii demonstrative
fișierele de iconițe (`vite.svg` sau `favicon.svg` și `icons.svg`, în funcție de versiune)	`frontend/public/`	iconițele paginii demonstrative
fișierul `index.css`	`frontend/src/`	stilurile paginii demonstrative
conținutul fișierului `App.css` (fișierul rămâne, gol)	`frontend/src/`	primește stilurile din Etapa 1 la Pasul 6
conținutul fișierului `App.jsx` (fișierul rămâne, gol)	`frontend/src/`	se rescrie la Pasul 8

Nu se șterg: `package.json`, `package-lock.json`, `vite.config.js`, `index.html`, `src/main.jsx`, `.gitignore`, `eslint.config.js` și folderul `node_modules/`.

După ștergere, pagina devine albă, iar consola browserului afișează o eroare, deoarece `main.jsx` încă importă fișierul `index.css`, care nu mai există (figura 3.8). Eroarea dispare după Pasul 7, fără repornirea serverului.

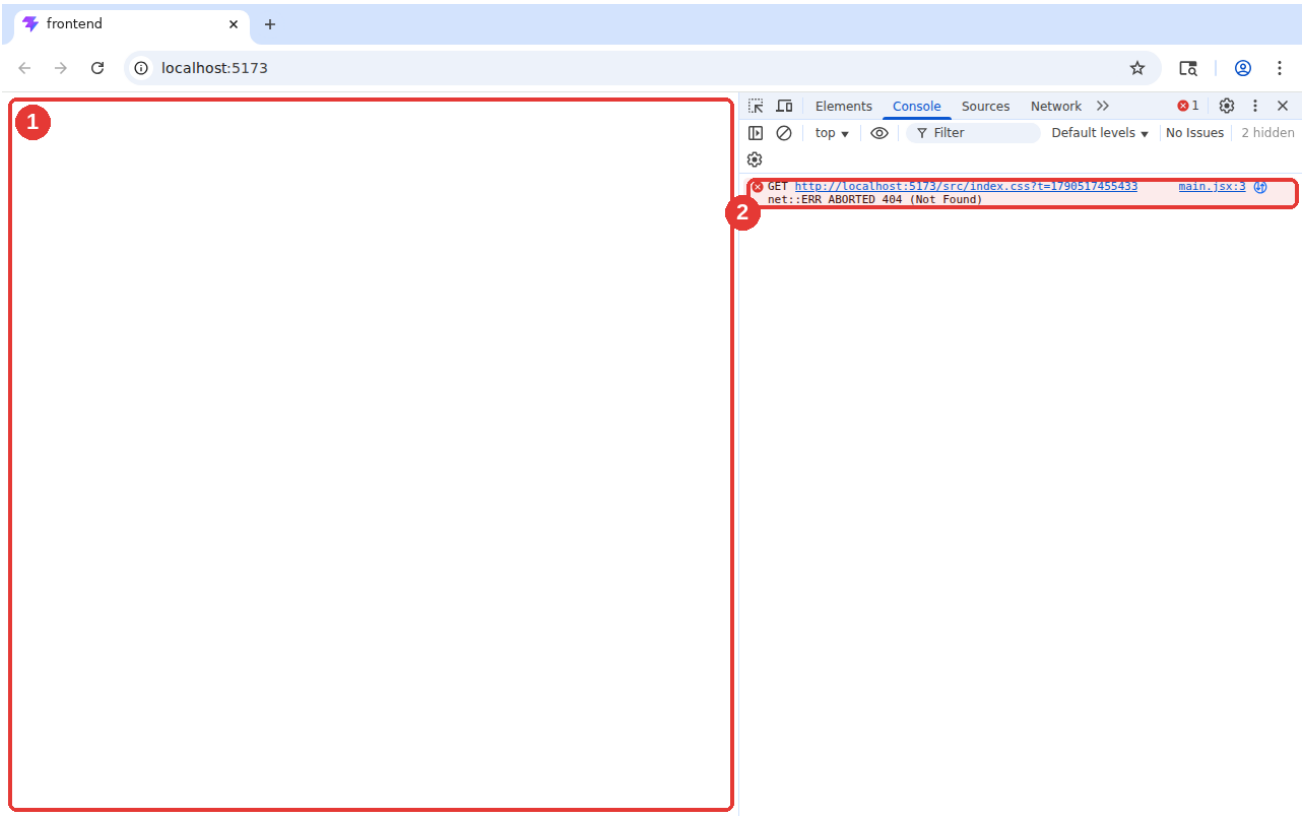


Fig. 3.8: Pagina după ștergerea fișierelor demonstrative, înainte de corectarea importurilor

1. Pagina albă: React nu mai are ce afișa.
2. Eroarea `404 (Not Found)` pentru `index.css`. În dreapta apare fișierul și linia care fac importul (`main.jsx:3`).

Pasul 6: Copierea stilurilor din Etapa 1

Conținutul fișierului `mockup/style.css` se copiază integral în `frontend/src/App.css`. În regula `.container` se păstrează o singură coloană, deoarece în această etapă există un singur panou; a doua coloană revine în Etapa 7, odată cu formularul:

```
.container {
  max-width: 1000px;
  margin: 24px auto;
  padding: 0 16px;
  display: grid;
  /* Etapa 3: doar panoul cu date; a doua coloană revine în Etapa 7 */
  grid-template-columns: 1fr;
  gap: 24px;
  align-items: start;
}
```

Pasul 7: Corectarea fișierului main.jsx

Fișierul `frontend/src/main.jsx` trebuie să aibă următorul conținut; practic, linia `import "./index.css";` se înlocuiește cu `import "./App.css";`:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./App.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
```

```
  </StrictMode>
);
```

Funcția `createRoot` găsește elementul `<div id="root">` din `index.html`, iar `render(<App />)` desenează acolo componenta `App`. `StrictMode` semnalează, doar în timpul dezvoltării, greșelile frecvente.

Pasul 8: Scrierea componentei App.jsx

Fișierul `App.jsx` se construiește treptat, cu salvare după fiecare secțiune.

8.1. Scheletul componentei. `export default` face funcția disponibilă pentru `main.jsx`. Ce se află după `return` ajunge pe ecran; fragmentul `<>...</>` grupează mai multe elemente fără a adăuga un element suplimentar în pagină.

```
export default function App() {
  return (
    <>
    </>
  );
}
```

8.2. Datele, deasupra funcției. Array-ul din Etapa 2 se copiază deasupra funcției `App`. React apelează funcția `App` la fiecare actualizare a ecranului; tot ce se află în interiorul ei este recreat de fiecare dată, în timp ce codul de deasupra se execută o singură dată. Pentru o listă fixă, locul corect este deasupra funcției.

8.3. Valoarea calculată. Pe primul rând din funcție, înainte de `return`, se calculează numărul de elemente active, cu aceeași logică din Etapa 2:

```
const active = taskuri.filter((t) => !t.gata).length;
```

8.4. Antetul. Între `<>` și `</>` se copiază antetul din mockup, cu două modificări: `class` devine `className`, iar numerele se scriu între acolade.

8.5. Datele brute, temporar. Sub antet se adaugă panoul care afișează datele ca text. `JSON.stringify` transformă array-ul în text, parametrul `2` stabilește indentarea, iar eticheta `pre` păstrează rândurile. Componenta completă:

```
const taskuri = [
  { id: 1, titlu: "Tema la Baze de date", gata: false, prioritate: "mare" },
  { id: 2, titlu: "Citit capitolul 3", gata: true, prioritate: "medie" },
  { id: 3, titlu: "Cumpărat cafea", gata: false, prioritate: "mica" },
];

export default function App() {
  const active = taskuri.filter((t) => !t.gata).length;

  return (
    <>
      <header className="app-header">
        <h1>TaskFlow</h1>
        <p>Organizează-ți task-urile pe categorii și priorități.</p>
        <p className="app-count">
          {taskuri.length} task-uri, dintre care {active} active
        </p>
      </header>
      <main className="container">
        <section className="panel">
          <h2>Task-urile mele</h2>
          {/* Temporar: date brute; componentele apar în Etapa 4 */}
          <pre>{JSON.stringify(taskuri, null, 2)}</pre>
        </section>
```

```
      </main>
    </>
  );
}
```

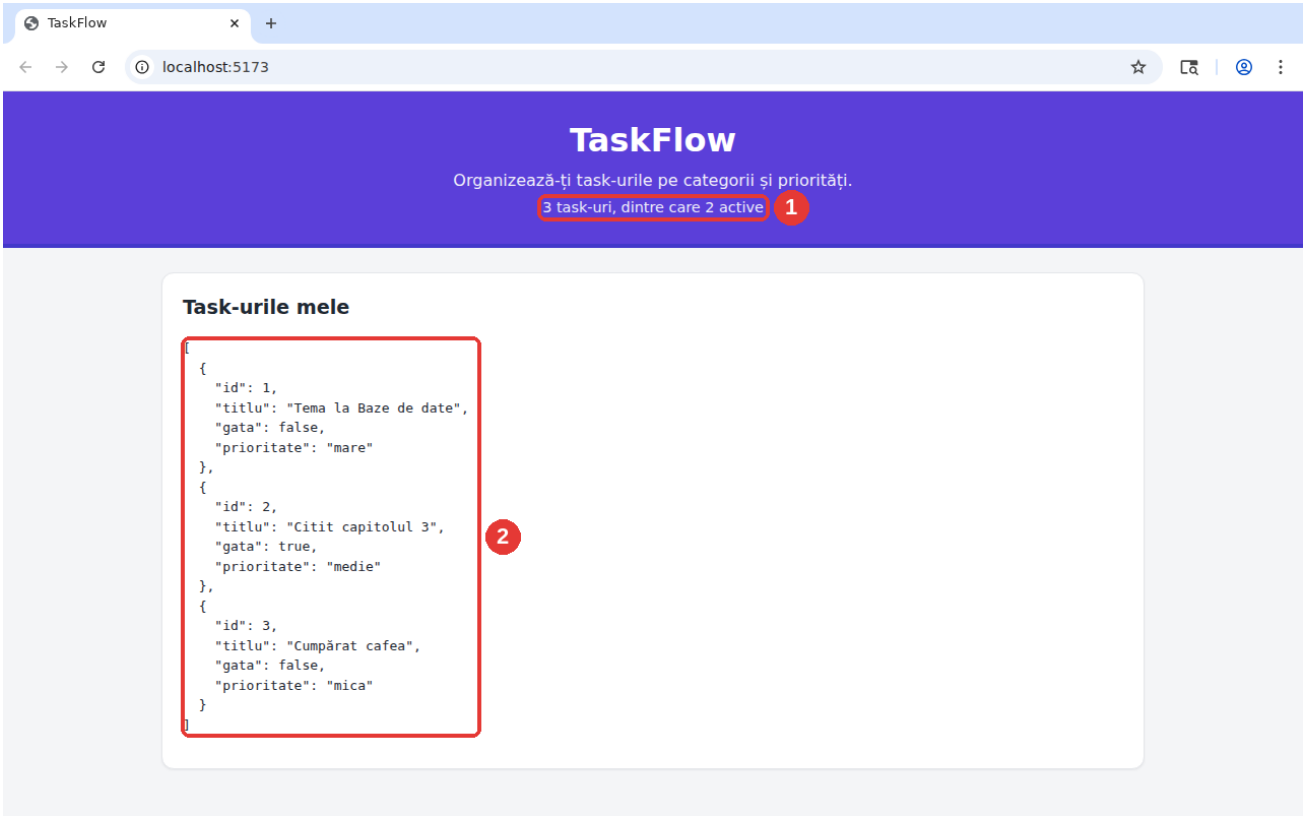


Fig. 3.9: Aplicația React la finalul Etapei 3

1. Contorul este calculat din date: `{taskuri.length}` și `{active}`. La modificarea unui element din array și salvarea fișierului, valorile se actualizează automat.
2. Lista afișată temporar ca JSON.

Pasul 9: Titlul paginii

În `frontend/index.html` se setează limba (`lang="ro"`) și titlul (`<title>TaskFlow</title>`), iar linia `<link rel="icon" ...>` se șterge, deoarece fișierul iconiței a fost eliminat la Pasul 5.

Pasul 10: Verificarea structurii și a build-ului

Într-un al doilea terminal, din rădăcina repository-ului, comanda `tree -I node_modules` (sau panoul *Explorer* din VS Code) trebuie să arate structura din figura 3.10.

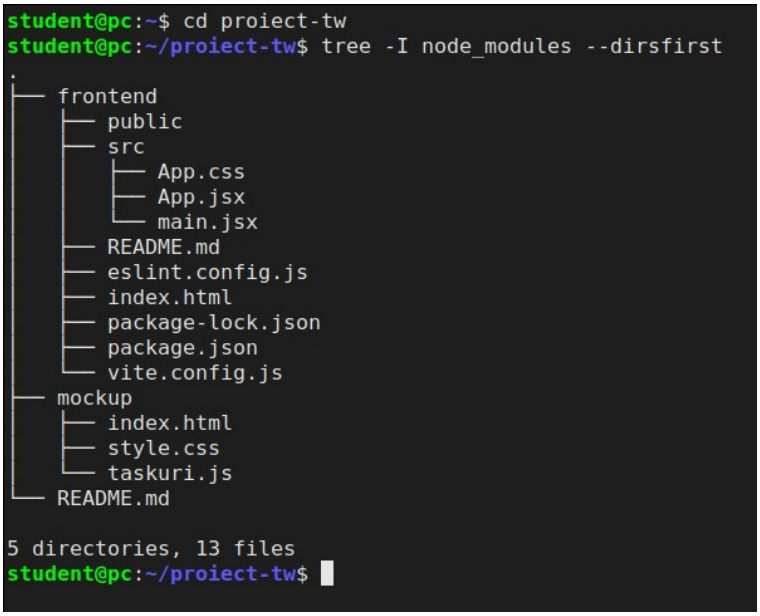


Fig. 3.10: Structura repository-ului la finalul Etapei 3

Din folderul `frontend/` se rulează `npm run build`, care împachetează aplicația în `frontend/dist/`. Nu se publică nimic; se verifică doar că proiectul se construiește fără erori (figura 3.11).

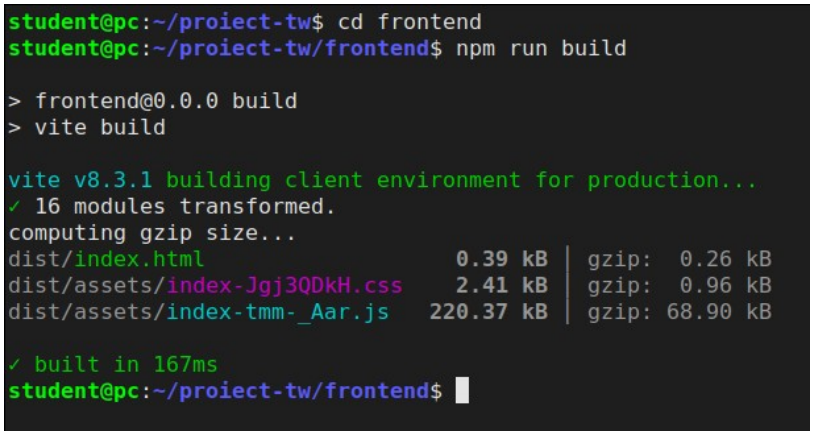


Fig. 3.11: Build fără erori

Pasul 11: README, jurnalul AI și publicarea pe GitHub

În README se actualizează secțiunea de rulare și stadiul:

```
## Stage 3: Vite and React project
The app now lives in `frontend/`, built with Vite. The data from stage 2
moved into the main component; the list is still printed raw.

## How to run
cd frontend
npm install
npm run dev
Then open the address printed in the terminal (http://localhost:5173).
```

Se completează `ai-log/etapa-03.md`. Înainte de commit, din rădăcina repository-ului, se verifică lista fișierelor care vor fi publicate (figura 3.12): nu trebuie să apară `node_modules/` sau `dist/`.

```
git add .
git status --short
git commit -m "Stage 3: Vite and React project"
git push
```

```
student@pc:~/proiect-tw/frontend$ cd ..
student@pc:~/proiect-tw$ git add .
student@pc:~/proiect-tw$ git status --short | head -20
A  frontend/.gitignore
A  frontend/README.md
A  frontend/eslint.config.js
A  frontend/index.html
A  frontend/package-lock.json
A  frontend/package.json
A  frontend/src/App.css
A  frontend/src/App.jsx
A  frontend/src/main.jsx
A  frontend/vite.config.js
R  index.html -> mockup/index.html
R  style.css -> mockup/style.css
R  taskuri.js -> mockup/taskuri.js
student@pc:~/proiect-tw$
```

Fig. 3.12: Fișierele incluse în commit; node_modules și dist sunt ignorate

Atenție! Dacă `git status` afișează sute de fișiere, folderul `node_modules/` nu este ignorat. Se verifică fișierul `.gitignore` din `frontend/`, apoi se rulează `git rm -r --cached node_modules` înainte de commit.

Tabelul de verificare din README

După publicarea codului, în README se adaugă tabelul de verificare al etapei (modelul din Etapa 1), publicat într-un commit separat, `Stage 3: checklist`.

Tabelul 3.4: Tabelul de verificare al Etapei 3

ID	Requirement	Where (permalink)	How to check
S3-R1	npm run dev starts without errors	frontend/package.json	npm install, npm run dev
S3-R2	app name and counter computed from data	frontend/src/App.jsx#L..-L..	edit data, save
S3-R3	data from README, not from the example	frontend/src/App.jsx#L..-L..	compare with README
S3-R4	stage 1 styles reused	frontend/src/App.css	look at the page
S3-R5	node_modules ignored, build passes	frontend/.gitignore	npm run build
S3-R6	README Stage 3 section + AI log	README.md, ai-log/etapa-03.md	read
S3-R7	commit “Stage 3” pushed	link to the commit	commit history

Lista de verificare a etapei

☐ **S3-R1** `npm run dev` pornește fără erori pe calculatorul studentului.

☐ **S3-R2** Pagina afișează numele aplicației și contorul calculat din date.

☐ **S3-R3** Datele sunt cele din README, nu cele din exemplu.

☐ **S3-R4** Stilurile din Etapa 1 se regăsesc în aplicație.

☐ **S3-R5** `node_modules/` nu apare pe GitHub, iar `npm run build` rulează fără erori.

☐ **S3-R6** README-ul este actualizat, iar `ai-log/etapa-03.md` este completat.

☐ **S3-R7** Commit-ul `Stage 3: ...` este publicat pe GitHub.

Probleme frecvente

Tabelul 3.5: Probleme frecvente și rezolvarea lor

Simptom	Cauză probabilă	Rezolvare
`npm: command not found`	Node.js nu este instalat	instalarea versiunii LTS și redeschiderea terminalului
`Cannot find module` la pornire	`npm install` nu a fost rulat în `frontend/`	`cd frontend`, apoi `npm install`
pagină albă, eroare 404 în consolă	un fișier demonstrativ a fost șters, dar importul lui a rămas	ștergerea liniei de import indicate în consolă (figura 3.8)
pagină albă, eroare de sintaxă	eroare de JSX (de obicei `class` sau o etichetă neînchisă)	mesajul din consolă sau din terminal indică fișierul și linia
portul 5173 este ocupat	un alt `npm run dev` rulează deja	oprirea lui cu Ctrl + C sau folosirea portului propus de Vite
modificările nu apar	fișierul editat nu este cel din `frontend/src`	verificarea căii fișierului deschis în editor

Cerințe până la etapa următoare

Până la etapa următoare, aplicația trebuie să pornească fără erori, iar codul, README-ul actualizat (cu tabelul de verificare) și jurnalul `ai-log/etapa-03.md` trebuie să fie publicate pe GitHub. În Etapa 4, lista afișată ca JSON este înlocuită de componente care desenează cardurile din date.